

Python for Traders — 2-Hour Session Guide

For Non-Coders | Goal: Vibe-Code Trading Scripts with AI

Instructor Note: Your learners don't need to become programmers. They need to read, tweak, and prompt AI to generate working trading scripts. Every concept below is taught through that lens.

Session Timeline at a Glance

Time	Part	Topic
0-15 min	Part 1	What Python Actually Is
15-35 min	Part 2	Variables — The Foundation
35-50 min	Part 3	Lists — Collections of Prices
50-70 min	Part 4	If Statements — The Heart of Strategy Logic
70-90 min	Part 5	Loops — What Backtesting Actually Is
90-105 min	Part 6	Functions — Reusable Blocks
105-115 min	Part 7	Pandas & NumPy — Your Data Power Tools
115-120 min	Part 8	Vibe Coding with ChatGPT
Bonus (if time)	Part 9	Mini Real Backtest Script Walkthrough

PART 1 — What Python Actually Is (0-15 min)

The Big Picture

Python is just a way of giving instructions to a computer in plain-ish English.

```
python  
  
print("Hello Traders")
```

That's it. One line. Run it. It does exactly what it says.

The Honest Truth About Your Goal

Tell your audience this upfront:

"You are NOT here to become a software engineer.
You're here to understand enough Python so you can tell ChatGPT what you want,
read what it gives you, and tweak it for your strategy."

The Real Workflow They'll Use

```
You describe a strategy idea
      ↓
ChatGPT writes the Python script
      ↓
You understand enough to modify it
      ↓
You test it → you change parameters → you iterate
```

Key Point to Drive Home

The only things they need to recognize:

- What a variable looks like
- What a list looks like
- What an if-condition looks like
- What a loop looks like
- What a function looks like

That's the entire vocabulary. The rest is ChatGPT's job.

PART 2 — Variables (15-35 min)

The Concept

A variable is just a labeled box that holds a value. That's it.

```
python

capital = 100000
risk_per_trade = 2
symbol = "NIFTY"
stop_loss_pct = 1.5
```

Draw this on screen/whiteboard:

capital	→	100000
--------------------	---	-------------------

risk_per_trade	→	2
---------------------------	---	--------------

symbol	→	"NIFTY"
-------------------	---	--------------------

Three Types They'll See

python

```
capital = 100000      # Number (integer) – no quotes
stop_loss = 1.5       # Number (decimal) – no quotes
symbol = "NIFTY"      # Text (string) – always has quotes
```

Rule of thumb: Text gets quotes. Numbers don't. That's 90% of what they need.

Variables Can Change

python

```
capital = 100000
print(capital)      # Output: 100000

capital = capital + 5000
print(capital)      # Output: 105000

capital = capital - 2000
print(capital)      # Output: 103000
```

This is literally how a backtest tracks portfolio value as trades happen.

Real Trading Example

python

```

capital = 500000
risk_per_trade = 2           # percentage
symbol = "BANKNIFTY"
entry_price = 45000
target_pct = 3
stop_loss_pct = 1

# Calculating position size
risk_amount = capital * risk_per_trade / 100
print(risk_amount)           # Output: 10000

# Calculating target and stop
target_price = entry_price + (entry_price * target_pct / 100)
stop_price = entry_price - (entry_price * stop_loss_pct / 100)

print(target_price)          # Output: 46350.0
print(stop_price)            # Output: 44550.0

```

Vibe Coding Prompt to show: "Create Python variables for a trading setup: capital of 5 lakh, NIFTY symbol, 2% risk per trade, 1.5% stop loss and 3% target. Then calculate the stop and target prices if entry is 19500."

PART 3 — Lists (35-50 min)

The Concept

A list is a collection of values stored in order. In trading, this is almost always a list of prices, returns, or signals.

```

python

prices = [100, 105, 110, 108, 120, 115, 125]

```

Accessing Items

Lists are numbered starting from 0 (yes, 0 — explain this once and move on).

```

python

```

```
prices = [100, 105, 110, 108, 120]
```

```
print(prices[0])    # Output: 100  (first item)
```

```
print(prices[2])    # Output: 110  (third item)
```

```
print(prices[-1])   # Output: 120  (last item – traders love this one)
```

Trading analogy: Think of it like candles on a chart. `prices[0]` is the oldest candle.

`prices[-1]` is the latest candle.

Adding to a List

```
python
```

```
prices = [100, 105, 110]
```

```
prices.append(115)
```

```
print(prices)        # Output: [100, 105, 110, 115]
```

This is how a live script would keep adding new prices as market data comes in.

Useful Built-in Tools for Traders

```
python
```

```
prices = [100, 105, 110, 108, 120]
```

```
print(len(prices))    # Output: 5 – how many candles/prices
```

```
print(max(prices))    # Output: 120 – highest price
```

```
print(min(prices))    # Output: 100 – lowest price
```

```
print(sum(prices))    # Output: 543 – total (useful for averages)
```

Real Trading Example

```
python
```

```
closing_prices = [19450, 19510, 19480, 19600, 19550, 19700, 19650]
```

```
highest = max(closing_prices)
lowest = min(closing_prices)
latest = closing_prices[-1]
previous = closing_prices[-2]
```

```
print("Resistance:", highest)    # Output: 19700
print("Support:", lowest)        # Output: 19450
print("Latest close:", latest)   # Output: 19650
print("Previous close:", previous) # Output: 19700
```

Keep it simple here. No slicing theory. No list comprehensions. They'll encounter those in AI-generated code — just tell them "that's advanced, ignore it for now, focus on the logic."

PART 4 — If Statements (50–70 min)

The Concept

This is the most important concept in all of trading logic. Every strategy, at its core, is:

```
IF [condition is true]
THEN [do this action]
ELSE [do that action]
```

Basic Syntax

```
python

price = 120

if price > 100:
    print("Price is above 100 — Bullish zone")
```

Note the indent. The indented line only runs IF the condition is True. Indentation = "this belongs inside the if block."

If / Else

```
python
```

```
price = 95

if price > 100:
    print("Bullish")
else:
    print("Bearish")
```

Output: Bearish

If / Elif / Else (Multiple Conditions)

```
python

rsi = 35

if rsi < 30:
    print("Oversold - Look for BUY")
elif rsi > 70:
    print("Overbought - Look for SELL")
else:
    print("Neutral zone")
```

Output: Neutral zone

Trading Strategy Example — The Real Thing

```
python
```

```

price = 19650
moving_average_20 = 19500
moving_average_50 = 19400
volume = 850000
avg_volume = 600000

if price > moving_average_20 and price > moving_average_50:
    print("Both MAs below price – Strong Bullish Signal")
elif price > moving_average_20:
    print("Above 20 MA only – Weak Bullish")
else:
    print("Below both MAs – Bearish")

if volume > avg_volume * 1.5:
    print("Volume confirmation: HIGH – Signal is stronger")
else:
    print("Volume low – Signal is weaker")

```

Comparison Operators (Just These 6)

```

python

# They'll see all of these in trading code
price > 100      # Greater than
price < 100      # Less than
price >= 100     # Greater than or equal
price <= 100     # Less than or equal
price == 100     # Equal to (note: TWO equal signs)
price != 100     # Not equal to

```

Combining Conditions

```

python

# and – BOTH must be true
if price > ma and rsi < 70:
    print("Buy signal")

# or – EITHER can be true
if price < support or rsi < 30:
    print("Oversold alert")

```


Tell them: "Most trading rules in code are just a bunch of if-statements chained together. That's all a strategy is."

PART 5 — Loops (70-90 min)

The Concept

A loop means: "Do this same thing for every item in a list."

Backtesting = looping through historical prices and checking your strategy condition at each candle.

```
python
prices = [100, 105, 110, 115, 120]

for price in prices:
    print(price)
```

Output:

```
100
105
110
115
120
```

Python goes through each price one by one and runs the indented code for each.

Loop + If = Basic Backtest

```
python
prices = [100, 105, 110, 108, 120, 115, 125]

for price in prices:
    if price > 112:
        print("Signal at price:", price)
```

Output:

Signal at price: 120
Signal at price: 115
Signal at price: 125

Tracking State Inside a Loop (This is Where Backtesting Happens)

python

```
capital = 100000
prices = [100, 105, 110, 108, 120, 115, 125]
entry_signal_price = 112
profit_per_signal = 500

for price in prices:
    if price > entry_signal_price:
        capital = capital + profit_per_signal
        print("Trade taken at:", price, "| Capital now:", capital)

print("Final capital:", capital)
```

Output:

```
Trade taken at: 120 | Capital now: 100500
Trade taken at: 115 | Capital now: 101000
Trade taken at: 125 | Capital now: 101500
Final capital: 101500
```

This IS backtesting. That simple loop is the skeleton of every backtest they'll ever run. The complexity later is just more conditions, more realistic P&L math — same core loop.

Real-ish Backtest Loop

python

```
capital = 500000
position = None
entry_price = 0
trades = 0
wins = 0

prices = [19450, 19510, 19480, 19600, 19550, 19700, 19650, 19800, 19750, 19900]
ma_value = 19550 # simplified fixed MA for demo

for price in prices:
    if position is None and price > ma_value:
        # Enter trade
        position = "LONG"
        entry_price = price
        print("BUY at:", entry_price)

    elif position == "LONG" and price > entry_price * 1.01: # 1% target
        # Exit trade
        profit = price - entry_price
        capital = capital + profit
        trades += 1
        wins += 1
        print("SELL at:", price, "| Profit:", profit, "| Capital:", capital)
        position = None

print("Total trades:", trades)
print("Final capital:", capital)
```

PART 6 — Functions (90-105 min)

The Concept

A function is a named, reusable block of code. Instead of writing the same calculation 10 times, you write it once as a function and call it by name.

Basic Syntax

```
python
```

```
def calculate_profit(buy_price, sell_price):  
    profit = sell_price - buy_price  
    return profit
```

`def` = "define a function" `calculate_profit` = the name you give it `buy_price, sell_price` = inputs (called parameters) `return` = what the function gives back

Using the Function

```
python  
  
profit = calculate_profit(19500, 19800)  
print(profit)           # Output: 300  
  
profit2 = calculate_profit(45000, 46500)  
print(profit2)          # Output: 1500
```

Trading Functions They'll Encounter

```
python  
  
def calculate_position_size(capital, risk_pct, entry, stop_loss):  
    risk_amount = capital * risk_pct / 100  
    risk_per_unit = entry - stop_loss  
    position_size = risk_amount / risk_per_unit  
    return round(position_size)  
  
# Usage  
size = calculate_position_size(  
    capital=5000000,  
    risk_pct=2,  
    entry=19650,  
    stop_loss=19500  
)  
print("Position size:", size, "units")  
# Output: Position size: 6666 units  
  
def is_bullish_candle(open_price, close_price):  
    return close_price > open_price  
  
print(is_bullish_candle(19500, 19650))    # Output: True  
print(is_bullish_candle(19700, 19550))    # Output: False
```

How to Read Any Function in AI-Generated Code

Teach them this 4-step read:

1. Find `def` — that's where the function starts
2. Read the name — it usually describes what it does
3. Look at the inputs in the parentheses
4. Find `return` — that's what comes out

python

```
# When you see this in ChatGPT code, read it like:  
# "This function is called 'simple_moving_average'  
# It takes a list of prices and a period (like 20)  
# It returns the average of the last 'period' prices"
```

```
def simple_moving_average(prices, period):  
    return sum(prices[-period:]) / period
```

PART 7 — Pandas & NumPy (105-115 min)

Why this matters: Every real trading and backtesting script uses Pandas and NumPy. This is where your next sessions live. Traders need to recognize these immediately.

NumPy — Fast Math on Lists

NumPy makes mathematical operations on large lists of numbers extremely fast — essential when working with years of historical price data.

Import it (they'll always see this at the top of scripts):

python

```
import numpy as np
```

Key NumPy things they'll see:

python

```

import numpy as np

prices = np.array([19450, 19510, 19480, 19600, 19700])

# NumPy does math on the whole list at once – no loop needed
print(np.mean(prices))    # Average: 19548.0
print(np.max(prices))     # Highest: 19700
print(np.min(prices))     # Lowest: 19450
print(np.std(prices))     # Standard deviation (volatility measure)

# Shift prices to calculate daily returns
returns = np.diff(prices) / prices[:-1] * 100
print(returns)            # % change between each candle

```

Why they'll use NumPy:

```

python

# Without NumPy – need a loop
returns = []
for i in range(1, len(prices)):
    r = (prices[i] - prices[i-1]) / prices[i-1] * 100
    returns.append(r)

# With NumPy – one line, much faster
prices_np = np.array(prices)
returns = np.diff(prices_np) / prices_np[:-1] * 100

```

Tell them: "You won't write NumPy from scratch. ChatGPT will. You just need to know:
`np.mean` = average, `np.std` = volatility, `np.array` = a fast version of a list."

Pandas — The Spreadsheet of Python

Pandas is essentially Excel inside Python. Every trading dataset — OHLCV data, indicators, signals — is handled with Pandas. It is the single most important library in trading Python.

Import it:

```

python

import pandas as pd

```

The DataFrame — a table of data:

python

```
import pandas as pd

# Creating a DataFrame (like an Excel sheet)
data = {
    'Date': ['2024-01-01', '2024-01-02', '2024-01-03', '2024-01-04', '2024-01-05'],
    'Open': [19400, 19500, 19450, 19600, 19550],
    'High': [19550, 19600, 19520, 19700, 19650],
    'Low': [19350, 19420, 19400, 19580, 19480],
    'Close': [19510, 19480, 19600, 19620, 19630],
    'Volume': [850000, 920000, 780000, 1100000, 960000]
}

df = pd.DataFrame(data)
print(df)
```

Output (looks like this):

	Date	Open	High	Low	Close	Volume
0	2024-01-01	19400	19550	19350	19510	850000
1	2024-01-02	19500	19600	19420	19480	920000
2	2024-01-03	19450	19520	19400	19600	780000
3	2024-01-04	19600	19700	19580	19620	1100000
4	2024-01-05	19550	19650	19480	19630	960000

Accessing columns — the most used thing:

python

```
print(df['Close'])           # All closing prices
print(df['Close'].iloc[-1])  # Latest closing price
print(df['Close'].iloc[0])    # First closing price
```

Calculating indicators directly on columns:

python

```
# Simple Moving Average – 3-period
df['SMA_3'] = df['Close'].rolling(window=3).mean()

# Exponential Moving Average – 3-period
df['EMA_3'] = df['Close'].ewm(span=3).mean()

# Daily returns
df['Returns'] = df['Close'].pct_change() * 100

print(df[['Date', 'Close', 'SMA_3', 'EMA_3', 'Returns']])
```

Creating signals — this is where strategy meets Pandas:

```
python

# Calculate 20-period SMA
df['SMA_20'] = df['Close'].rolling(window=20).mean()

# Create Buy signal: price crosses above SMA
df['Signal'] = 0 # default = no signal
df.loc[df['Close'] > df['SMA_20'], 'Signal'] = 1 # 1 = Buy
df.loc[df['Close'] < df['SMA_20'], 'Signal'] = -1 # -1 = Sell

print(df[['Date', 'Close', 'SMA_20', 'Signal']])
```

Loading real data from a CSV file:

```
python

# This is what they'll actually do in the course
df = pd.read_csv("NIFTY_data.csv")
print(df.head()) # See first 5 rows
print(df.tail()) # See last 5 rows
print(df.shape) # How many rows and columns
print(df.columns) # Column names
print(df.describe()) # Basic stats on all columns
```

Filtering data — like Excel filter:

```
python
```



```
# Only show rows where volume is high
high_volume_days = df[df['Volume'] > 1000000]

# Only show rows where close is above SMA
bullish_days = df[df['Close'] > df['SMA_20']]

# Date range filter
jan_data = df[df['Date'] >= '2024-01-01']
```

The Pandas Cheat Sheet (Print This for Students)

```
python

# READ DATA
df = pd.read_csv("file.csv")          # Load data
df.head()                             # First 5 rows
df.tail()                             # Last 5 rows
df.describe()                         # Summary stats

# ACCESS DATA
df['Close']                           # A column
df['Close'].iloc[-1]                  # Last value
df['Close'].iloc[0]                   # First value

# CALCULATIONS
df['Close'].mean()                    # Average
df['Close'].max()                     # Highest
df['Close'].min()                     # Lowest
df['Close'].rolling(20).mean()        # 20-period moving average
df['Close'].pct_change()              # % change per row
df['Close'].cumsum()                  # Cumulative sum
df['Close'].diff()                    # Difference from previous

# INDICATORS (just a column operation)
df['SMA20'] = df['Close'].rolling(20).mean()
df['EMA20'] = df['Close'].ewm(span=20).mean()
df['Returns'] = df['Close'].pct_change() * 100

# FILTERING
df[df['Close'] > 19500]                # Where close > 19500
df[df['Signal'] == 1]                 # Where signal is Buy
```

Tell them: "You will not memorize all of this. You will copy-paste from this sheet or ask ChatGPT. What matters is you recognize `df`, `df['column']`, and `.rolling().mean()` when you see them in a script — so you know what to change."

PART 8 — Vibe Coding with ChatGPT (115-120 min)

What Vibe Coding Is

Vibe coding = describing what you want in plain English, getting code from AI, reading it enough to tweak it, running it, iterating.

You are the trader. ChatGPT is the programmer. Your job is to give clear instructions.

The Prompts That Work

Creating a strategy:

Create a Python script that:

- Takes a list of daily closing prices for NIFTY
- Calculates 20-period and 50-period Simple Moving Average
- Prints "BUY SIGNAL" when the 20 SMA crosses above the 50 SMA
- Prints "SELL SIGNAL" when the 20 SMA crosses below the 50 SMA
- Use Pandas for all calculations
- Use a sample dataset of 60 prices in the script

Modifying a script:

Change this script to use a 50 SMA and 200 SMA instead of 20 and 50.
Also change the capital from 100000 to 500000.
[paste the script]

Adding risk management:

Add to this strategy:

- 1.5% stop loss from entry price
- 3% profit target from entry price
- Maximum 2 trades open at a time

[paste the script]

Explaining code:

Explain this Python script line by line in simple English.
I am a trader, not a programmer – use trading terminology where possible.
[paste the script]

Debugging:

This script gives an error: [paste error]
Here is the script: [paste script]
Please fix it and explain what was wrong.

Tweaking parameters:

In this backtest script, I want to:

1. Change the moving average from 20 to 14 periods
2. Change risk per trade from 2% to 1%
3. Add a print statement showing the win rate at the end

[paste the script]

The Golden Rule of Vibe Coding

The more specific your prompt, the better the code.

Vague: *"Write a moving average strategy"* Good: *"Write a Python script using Pandas that loads OHLCV data from a CSV, calculates 20 and 50 period SMA on the Close column, generates buy/sell signals when they cross, and prints a summary showing total trades, wins, losses, and final capital starting at 5 lakh with 1000 profit per trade."*

BONUS: Mini Real Backtest Walkthrough (If time permits)

Show this script and teach them what to tweak

python

```

import pandas as pd
import numpy as np

# =====
# PARAMETERS — THESE ARE THE KNOBS TO TURN
# =====
CAPITAL = 500000          # Starting capital in INR
RISK_PER_TRADE_PCT = 2    # Risk 2% of capital per trade
SMA_FAST = 20             # Fast moving average period
SMA_SLOW = 50             # Slow moving average period
PROFIT_TARGET_PCT = 3     # Take profit at 3%
STOP_LOSS_PCT = 1.5      # Stop loss at 1.5%

# =====
# LOAD DATA
# =====
# In real use: df = pd.read_csv("NIFTY_data.csv")
# For demo, we create sample data
np.random.seed(42)
dates = pd.date_range(start='2023-01-01', periods=100, freq='B')
close_prices = 19000 + np.cumsum(np.random.randn(100) * 50)

df = pd.DataFrame({'Date': dates, 'Close': close_prices})

# =====
# CALCULATE INDICATORS
# =====
df['SMA_Fast'] = df['Close'].rolling(SMA_FAST).mean()
df['SMA_Slow'] = df['Close'].rolling(SMA_SLOW).mean()

# Generate crossover signals
df['Signal'] = 0
df.loc[df['SMA_Fast'] > df['SMA_Slow'], 'Signal'] = 1  # Bullish
df.loc[df['SMA_Fast'] < df['SMA_Slow'], 'Signal'] = -1 # Bearish

# =====
# SIMPLE BACKTEST LOOP
# =====
capital = CAPITAL
position = None
entry_price = 0
trades = []

```

```

for i in range(1, len(df)):
    row = df.iloc[i]
    prev_row = df.iloc[i - 1]

    # Entry: crossover happened
    if prev_row['Signal'] != 1 and row['Signal'] == 1 and position is None:
        position = 'LONG'
        entry_price = row['Close']
        target = entry_price * (1 + PROFIT_TARGET_PCT / 100)
        stop = entry_price * (1 - STOP_LOSS_PCT / 100)

    # Exit: target or stop hit
    if position == 'LONG':
        if row['Close'] >= target:
            pnl = target - entry_price
            capital += pnl
            trades.append({'type': 'WIN', 'pnl': pnl})
            position = None
        elif row['Close'] <= stop:
            pnl = stop - entry_price
            capital += pnl
            trades.append({'type': 'LOSS', 'pnl': pnl})
            position = None

# =====
# RESULTS SUMMARY
# =====
total_trades = len(trades)
wins = len([t for t in trades if t['type'] == 'WIN'])
losses = total_trades - wins
win_rate = (wins / total_trades * 100) if total_trades > 0 else 0
total_pnl = capital - CAPITAL

print("=" * 40)
print("          BACKTEST RESULTS")
print("=" * 40)
print(f"Starting Capital : ₹{CAPITAL:,.0f}")
print(f"Final Capital      : ₹{capital:,.0f}")
print(f"Total P&L          : ₹{total_pnl:,.0f}")
print(f"Total Trades       : {total_trades}")
print(f"Wins               : {wins}")
print(f"Losses             : {losses}")

```

```
print(f"Win Rate           : {win_rate:.1f}%")
print("=" * 40)
```

What to Show Them to Change

Point to the PARAMETERS block and say:

"This is the only section you usually need to touch. These are your knobs. Change `SMA_FAST` from 20 to 14. Change `STOP_LOSS_PCT` from 1.5 to 2. Run the script again. See if the results improve. That's backtesting. That's the whole loop."

What NOT to Teach (Save Everyone's Time)

These topics add zero value for trading script users:

- ❌ Classes and Object-Oriented Programming
- ❌ Decorators, Generators, Lambda
- ❌ Virtual environments and packaging
- ❌ Recursion and algorithms
- ❌ Exception handling (deep dive)
- ❌ Data structures theory
- ❌ Complexity analysis
- ❌ Module architecture

If a student asks about these — *"That's advanced Python. ChatGPT handles that. Focus on reading the strategy logic."*

One-Page Cheat Sheet for Students

VARIABLES	capital = 100000	# a labeled box with a value
TEXT VARIABLE	symbol = "NIFTY"	# text always has quotes
LIST	prices = [100,105,110]	# collection of values
ACCESS LIST	prices[0]	# first item
ACCESS LAST	prices[-1]	# last item
IF CONDITION	if price > ma:	# check a condition
	print("Buy")	# do this if true
	else:	
	print("Sell")	# do this if false

LOOP	for price in prices:	# repeat for each item
	print(price)	# runs once per price
FUNCTION	def my_func(x, y):	# define reusable block
	return x + y	# output
IMPORT PANDAS	import pandas as pd	
LOAD CSV	df = pd.read_csv("file.csv")	
GET COLUMN	df['Close']	
MOVING AVG	df['Close'].rolling(20).mean()	
IMPORT NUMPY	import numpy as np	
AVERAGE	np.mean(prices)	
STD DEV	np.std(prices)	# volatility
VIBE CODE:	Describe strategy → ChatGPT writes → you tweak → run → iterate	